

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

**Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий**

Отчёт по заданию №1. Тестирование сайтов

Направление подготовки 01.03.02 Прикладная математика и информатика
курс 3

Направленность (профиль) Прикладная математика

Работу выполнил: И.И. Иванов

Работу проверил: А.А. Полупанов

Краснодар

2026

Задание (темы 1, 2, см. слайды 1-7): протестировать сайты <https://www.sportmaster.ru> и <https://kubsu.ru> согласно приведённому чеклисту. Результат оформить в виде отчёта.

Тестирование веб-сайта <https://kubsu.ru>

Включает в себя:

- 1) Documentation Testing;
- 2) Functionality Testing;
- 3) GUI Testing;
- 4) Usability Testing;
- 5) Interface Testing;
- 6) Database Testing;
- 7) Compatibility Testing;
- 8) Performance Testing;
- 9) Security Testing;
- 10) Crowd Testing.

1. Documentation Testing (нет возможности протестировать)

Плохая документация может повлиять на качество продукта. Хорошая документация по продукту играет решающую роль в конечном продукте. Таким образом, тестирование документации играет жизненно важную роль в тестировании программного обеспечения. Тестирование задокументированных артефактов, разработанных до, во время и после тестирования продукта, называется тестированием документации. Ниже приведены некоторые часто используемые артефакты:

- Requirement documents;
- Test Plan;
- Test Cases;
- Traceability Matrix (RTM).

Подробнее о тестировании документации написано в видах тестирования.

2. Functionality Testing

Функциональное тестирование - тестирование того, что делает система. Проверить, что каждая функция программного приложения ведет себя так, как указано в документе с требованиями. Тестирование всех функций путем предоставления соответствующих входных данных, чтобы проверить, соответствует ли фактический результат ожидаемому результату. Оно используется для проверки рабочих процессов, всех ссылок веб-страниц,

тестирования форм, тестирования файлов cookie и подключения к базе данных. Обычно функциональное тестирование включает в себя следующие задачи:

- **Testing UI Workflows:** тестируются end to end workflow или бизнес-сценарии. Рекомендуется написание тестовых сценариев или тестовых случаев, чтобы охватить различные сценарии и установить критерии прохождения;
- **Тестирование гиперссылок:** все ссылки на веб-сайте работают правильно, и нет неработающих ссылок. Типы ссылок включают внутренние ссылки, исходящие ссылки, якорные ссылки, схемы mailto и т. д.;
- **Тестирование форм** (проверка полей ввода): формы используются для интерактивного общения с конечными пользователями. Тестировщик должен убедиться, что все формы работают должным образом. Тестирование форм включает в себя:
 - заполняются ли значения по умолчанию;
 - отображается ли сообщение об ошибке, когда пользователь не заполняет обязательное поле;
 - принимает ли форма недопустимые значения;
 - формы оптимально отформатированы для лучшей читабельности;
 - поля AJAX правильно заполняют значения во время выполнения;
 - загружаются ли раскрывающиеся списки с параметрами.
- **Проверка файлов cookie:** подробно о тестировании кук написано в теме про cookie в сетях;
- **Проверка HTML и CSS:** Тестировщик должен проверить, имеет ли сайт чистую структуру HTML и оптимизированный CSS в соответствии со стандартами W3C. Также нужно убедиться, что поисковые системы могут легко сканировать сайт.
 - нет синтаксических ошибок HTML;
 - цветовые схемы читаемы;
 - карта сайта точна.

Примеры функциональных тест-кейсов:

- Кнопки:
 - Enter должна срабатывать как submit; **Успешно**
 - Tab должен переводить курсор на следующий элемент. **Частично Успешно**
 - ✦ Tab переводит курсор на следующий элемент, но никак не отображается то место, на котором стоит Tab.
- Поля ввода:

○ trimming («убирание») пробелов в полях ввода; **Неудачно** ○ пустота/пробелы в поле ввода; **Неудачно** ○ все способы редактирования (Insert, Delete, Backspace, Ctrl+C/V/X/Z и т. д.);

Частично Успешно ○
дроби (1.5/ 1,5/ ½). **Неудачно**

3. GUI

Верстка - размещение элементов веб-приложения (изображения, текст, кнопки, видео...) в соответствии с макетом или требованиями. Проверяем:

- наличие всех элементов; **Успешно**
- их размер и цвет; **Частично Успешно**
 - ✦ Сайт выполнен в единообразной цветовой палитре, но размеры и стили шрифтов часто не совпадают и контрастируют друг на фоне друга
- расположение относительно друг-друга. **Неудачно**
 - ✦ Как пример расположение эмблемы сайта с его названием расположена не вровень с другими элементами меню, а чуть ниже

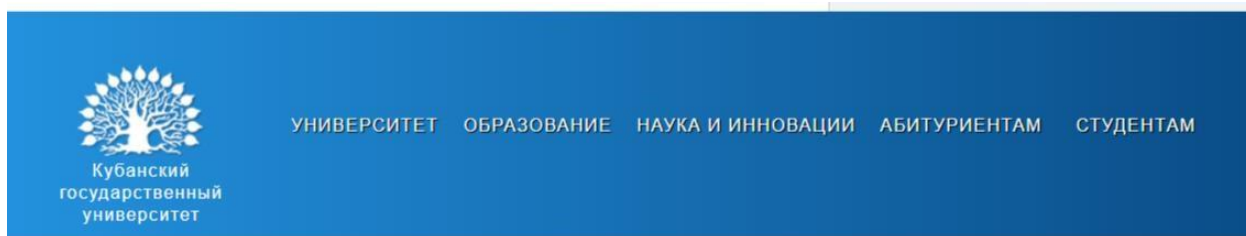


Рис .3

- Сравнение с макетом - метод наложения готового эталонного макета (обычно psd-файл) на приложение в экране браузера, все несовпадения можно рассматривать как ошибки (для этого есть хороший инструмент **Pixel Perfect**).
- Измерение размеров элемента - если это имеет значение, то померять размеры элемента и сравнить их со спецификацией можно с помощью, например **Page Ruler**.
- Правильность шрифтов (название, размер, цвет) - **WhatFont**.
- Цвета интерфейса - **ColorZilla**.
- Контент - проверить на наличие орфографических и грамматических ошибок (**SpellChecker**). **Успешно**

- Появление курсора - довольно часто мы забываем проверить, появляется ли вообще и как выглядит курсор в полях ввода, на кликабельных элементах. **Успешно**
- Фавикон - такая маленькая незначительная вещица, но может изрядно подпортить впечатление пользователя (в моей практике были случаи, когда разработчики или дизайнеры шаблона оставляли фавикон с логотипом своей компании на сайте у заказчика). **Успешно**
- Обозначение возможности переноса элементов.
- Кодировка (UTF8...). **Успешно**
- Стандарты HTML/CSS - достаточно неплохие решения для быстрой проверки предлагает **W3C**.

4. Usability Testing

Юзабилити-тестирование стало важной частью любого веб-проекта. Его могут провести тестировщики или небольшая фокус-группа, похожая на целевую аудиторию вебприложения чтобы проверить, является ли приложение удобным для пользователя, и было ли комфортно использовать его конечному пользователю:

Примеры юзабилити тест-кейсов:

- Текст подсказки должен быть там для каждого поля. **Успешно**
- Домашняя ссылка должна быть на каждой странице. **Успешно**
- Сообщение о подтверждении должно отображаться для любого вида операции обновления и удаления.
- Полоса прокрутки должна появляться только при необходимости. **Неудачно**
- Если при отправке появляется сообщение об ошибке, информация, заполненная пользователем, должна быть там. **Успешно**
- Название должно отображаться на каждой веб-странице. **Успешно**
- Все поля (текстовое поле, раскрывающийся список, переключатель и т. д.) и кнопки должны быть доступны с помощью сочетаний клавиш, и пользователь должен иметь возможность выполнять все операции с помощью клавиатуры. **Частично Успешно** ✦
В ряде случаев нет отображения того места, на котором на данный момент находится пользователь

5. Interface Testing Тестирование интерфейсов предназначено для проверки интерфейса между вебсервером и сервером приложений, правильно ли взаимодействуют сервер приложений и сервер баз данных. Это гарантирует положительный пользовательский опыт. Он включает в себя проверку процессов связи, а также проверку правильности отображения сообщений об ошибках.

- Приложение: тестовые запросы правильно отправляются в базу данных и вывод на стороне клиента отображается правильно. Ошибки, если таковые имеются, должны

быть обнаружены приложением и должны отображаться только администратору, а не конечному пользователю;

- Веб-сервер: тестовый веб-сервер обрабатывает все запросы приложений без какого-либо отказа в обслуживании; **Успешно**

✦ Сервер успешно отвечает на все запросы без каких-либо нареканий

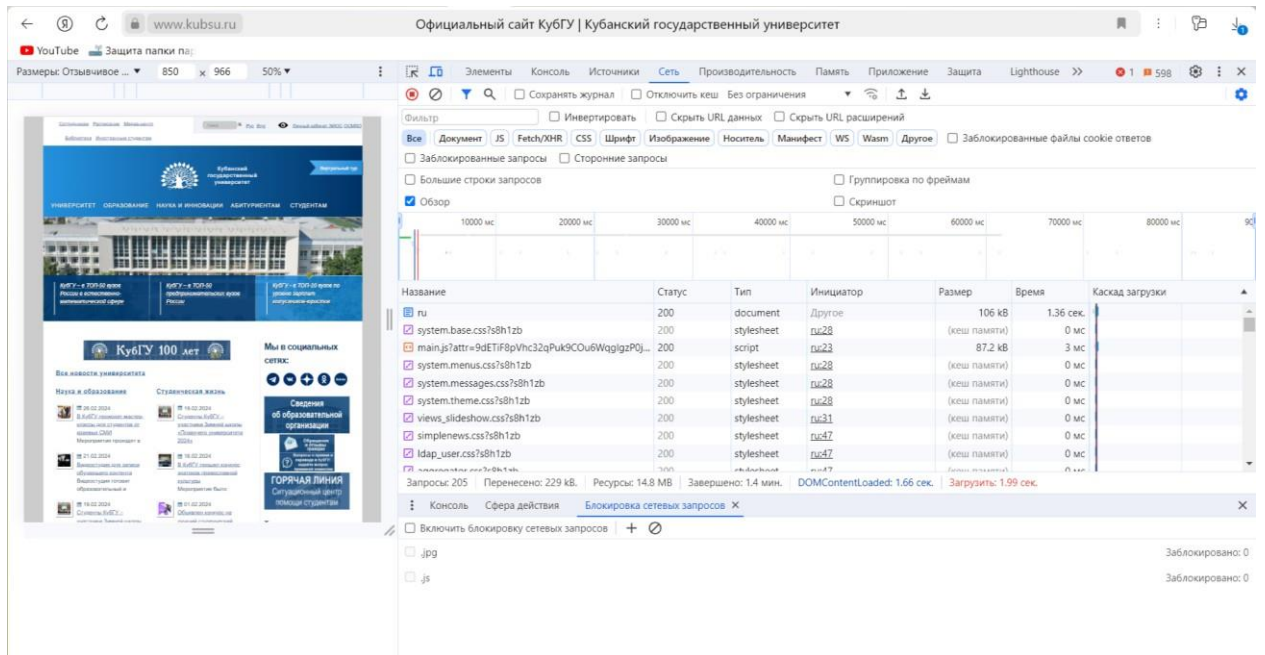


Рис. 4

✦ Также были проведены выборочные тестовые запросы

Overview GET Request No Environment

HTTP cbr / Request Save Send

GET -- https://www.kubsu.ru/modules/system/system.menus.css?s8h1zb\

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Description	*** Bulk Edit
☒	s8h1zb\			
	Key	Value	Description	

Body Cookies Headers (19) Test Results Status: 200 OK Time: 28 ms Size: 1.26 KB Save as Example

Pretty Raw Preview Visualize Text

```
1
2 /**
3  * @file
4  * Styles for menus and navigation markup.
5  */
6
7 /**
8  * Markup generated by theme_menu_tree().
9  */
10 ul.menu {
11   border: none;
12   list-style: none;
```

Postbot Runner Capture requests Cookies Trash

Рис. 5

Overview GET Request No Environment

HTTP cbr / Request Save Send

GET https://www.kubsu.ru/misc/jquery-html-prefilter-3.5.0-backport.js?v=1.7.2

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Description	*** Bulk Edit
☒	v	1.7.2		
	Key	Value	Description	

Body Cookies Headers (17) Test Results Status: 200 OK Time: 95 ms Size: 4.94 KB Save as Example

Pretty Raw Preview Visualize Text

```
1 /**
2  * For jQuery versions less than 3.5.0, this replaces the jQuery.htmlPrefilter()
3  * function with one that fixes these security vulnerabilities while also
4  * retaining the pre-3.5.0 behavior where it's safe to do so.
5  * - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-11022
6  * - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-11023
7  *
8  * Additionally, for jQuery versions that do not have a jQuery.htmlPrefilter()
9  * function (1.x prior to 1.12 and 2.x prior to 2.2), this adds it, and
10 * extends the functions that need to call it to do so.
11 *
12 * Drupal core's jQuery version is 1.4.4, but jQuery Update can provide a
```

Postbot Runner Capture requests Cookies Trash

Рис. 6

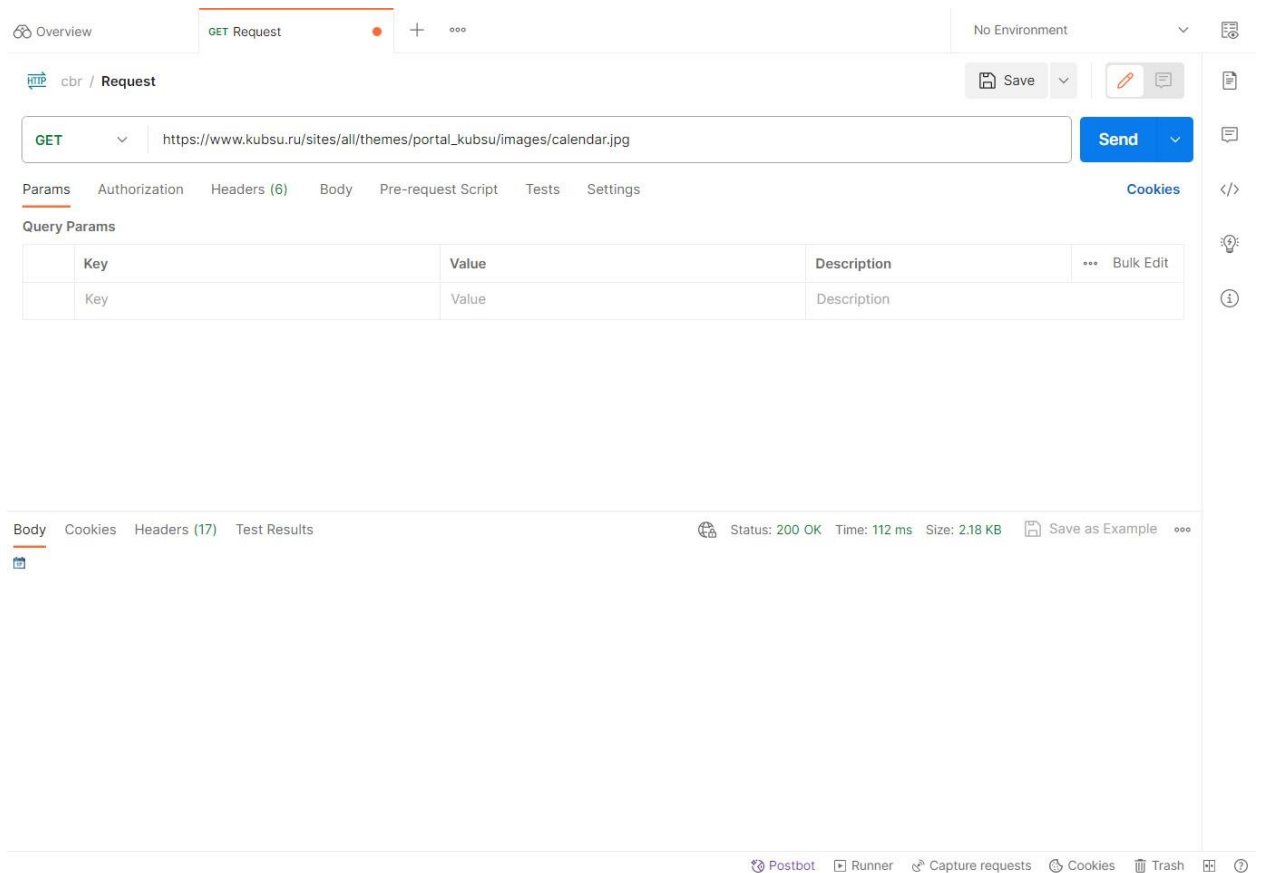


Рис. 7

- Сервер базы данных: запросы, отправленные в базу данных, дают ожидаемые результаты. Проверьте реакцию системы, когда невозможно установить соединение между тремя уровнями (Приложение, Интернет и База данных) и соответствующее сообщение отображается конечному пользователю.

6. Database Testing

Тестирование баз данных (back-end тестирование, тестирование данных) включает проверку целостности данных на front end с данными на back end. Оно проверяет схему, таблицы базы данных, столбцы, индексы, хранимые процедуры, триггеры, дублирование данных, потерянные записи, ненужные записи. Оно включает в себя обновление записей в базе данных и их проверку на внешнем интерфейсе. Тестирование будет включать в себя:

- Отображение ошибок при выполнении запросов;
- Целостность данных поддерживается при создании, обновлении или удалении данных в базе данных;
- Тестирование производительности базы данных;
- Тестирование процедур, триггеров и функций.

7. Compatibility Testing

Тестирование совместимости предназначено для проверки совместимости приложения в разных браузерах и на различных устройствах.

- Тестирование совместимости браузера: кросс-браузерное тестирование - это тип нефункционального теста, который помогает нам убедиться, что наш веб-сайт или веб-приложение работают должным образом в различных веб-браузерах. При тестировании веб-сайта нам необходимо убедиться, что он отображается одинаково во всех браузерах. Нам нужно предоставить одинаковый опыт для пользователей, независимо от того, какой тип ОС и какой браузер они используют. Не все используют одну и ту же среду. Несмотря на то, что Google Chrome является самым популярным на текущем рынке, все же множество пользователей используют Mozilla Firefox, Safari и другие. Если веб-сайт не работает должным образом в конкретном браузере, это ухудшает взаимодействие с пользователем. Нужно проверить, правильно ли отображается ваше веб-приложение в браузерах, работает ли JavaScript, AJAX и аутентификация. Вы также можете проверить рендеринг веб-элементов, таких как кнопки, текстовые поля и т. д.

Примеры тестов на совместимость:

- Протестируйте сайт в разных браузерах (IE, Firefox, Chrome, Safari и Opera) и убедитесь, что сайт отображается правильно. **Успешно**
- Используемая версия HTML совместима с соответствующими версиями браузера. **Успешно**

✦ На данном сайте используется версия HTML5. Все браузеры, в которых тестировался сайт, совместимы с этой версией HTML.

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML+RDFa 1.0//EN"
2   "http://www.w3.org/MarkUp/DTD/xhtml-rdfa-1.dtd">
3 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="ru" version="XHTML+RDFa 1.0" dir="ltr"
4   xmlns:content="http://purl.org/rss/1.0/modules/content/"
5   xmlns:dc="http://purl.org/dc/terms/"
6   xmlns:foaf="http://xmlns.com/foaf/0.1/"
7   xmlns:og="http://ogp.me/ns#"
8   xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
9   xmlns:sioc="http://rdfs.org/sioc/ns#"
10  xmlns:sioc="http://rdfs.org/sioc/types#"
11  xmlns:skos="http://www.w3.org/2004/02/skos/core#"
12  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
13  xmlns:owl="http://www.w3.org/2002/07/owl#"
14  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
15  xmlns:rss="http://purl.org/rss/1.0/"
```

Рис. 8

- ✦ Версии браузеров в которых проводилось сравнительное тестирование:

Яндекс Браузер : 24.1.1.928 (64-bit)

Microsoft Edge : Версия 122.0.2365.52 (Официальная сборка)
(64разрядная версия)

Google Chrome : Версия 122.0.6261.70 (Официальная сборка), (64 бит)

Firefox Browser : 123.0 (64-разрядный)

- Проверьте правильность отображения изображений в разных браузерах. **Успешно**
- Протестируйте шрифты, которые можно использовать в разных браузерах. **Успешно**
- Протестируйте код Javascript в разных браузерах. **Успешно**
- Проверьте анимированные GIF-файлы в разных браузерах.

8. Performance Testing

Тестирование производительности определяет или подтверждает характеристики скорости, масштабируемости и/или стабильности тестируемой системы или приложения. Производительность связана с достижением времени отклика, пропускной способности и уровня использования ресурсов, которые соответствуют целям производительности для проекта или продукта. Тестирование производительности веб-приложений проводится для снижения риска доступности, надежности, масштабируемости, быстродействия, стабильности и т. д. системы. Тестирование производительности включает в себя ряд различных типов тестирования, таких как нагрузочное тестирование, объемное тестирование, стресс-тестирование, тестирование емкости, тестирование выдержки/выносливости и пиковое тестирование, каждое из которых предназначено для выявления или решения проблем с производительностью в системе.

Примеры тестов:

- Имитируем нагрузку пользователями (JMeter);
- Пробуем загрузить большие объемы данных, файлы, медиа;
- Нагружаем БД;
- Понижаем скорость инета (NetLimiter); **Частично Успешно**

- ✦ При ограничении интернета в скорости 3G (высокая скорость) в DevTools время выполнения 190 запросов равно 15.21 секунд, при отсутствии ограничения скорости – 12.60 секунд.

9. Security Testing

Тестирование безопасности - это процесс, позволяющий определить, защищает ли система данные и поддерживает ли она функциональность, как предполагалось. Тестирование безопасности направлено на выявление всех возможных лазеек и слабых мест системы на самом начальном этапе, чтобы избежать нестабильной работы системы,

неожиданного сбоя, потери информации, потери дохода, потери доверия клиентов. Тесты безопасности включают тестирование на наличие уязвимостей, таких как:

- SQL-инъекция (SQL Injection);
- Межсайтовый скриптинг (XSS);
- Управление сеансом (Session Management);
- Сломанная аутентификация;
- Подделка межсайтовых запросов (CSRF);
- Неправильная конфигурация безопасности;
- Невозможность ограничить доступ к URL-адресу;
- Раскрытие секретных данных;
- небезопасная прямая ссылка на объект;
- Отсутствует контроль доступа на функциональном уровне;
- Использование компонентов с известными уязвимостями;
- Непроверенные перенаправления и возвраты.

10. Crowd Testing or Crowdsourced testing

Краутестинг или краудсорсинговое тестирование - это новая тенденция в тестировании программного обеспечения, которая использует толпу (crowd/большое количество людей) для выполнения тестов, которые в противном случае были бы выполнены выбранной группой людей в компании. Краудсорсинговое тестирование представляет собой интересную и перспективную концепцию и помогает выявить многие незамеченные дефекты. Оно включает в себя практически все типы тестирования, применимые к вашему веб-приложению.

Тестирование веб-сайта <https://www.sportmaster.ru>

Включает в себя:

- 1) Documentation Testing;
- 2) Functionality Testing;
- 3) GUI Testing;
- 4) Usability Testing;
- 5) Interface Testing;
- 6) Database Testing;
- 7) Compatibility Testing;
- 8) Performance Testing;
- 9) Security Testing; 10) Crowd Testing.

1. Documentation Testing (нет возможности протестировать)

Плохая документация может повлиять на качество продукта. Хорошая документация по продукту играет решающую роль в конечном продукте. Таким образом, тестирование документации играет жизненно важную роль в тестировании программного обеспечения. Тестирование задокументированных артефактов, разработанных до, во время и после тестирования продукта, называется тестированием документации. Ниже приведены некоторые часто используемые артефакты:

Подробнее о тестировании документации написано в видах тестирования.

2. Functionality Testing

Функциональное тестирование - тестирование того, что делает система. Проверить, что каждая функция программного приложения ведет себя так, как указано в документе с требованиями. Тестирование всех функций путем предоставления соответствующих входных данных, чтобы проверить, соответствует ли фактический результат ожидаемому результату. Оно используется для проверки рабочих процессов, всех ссылок веб-страниц, тестирования форм, тестирования файлов cookie и подключения к базе данных. Обычно функциональное тестирование включает в себя следующие задачи:

- **Testing UI Workflows:** тестируются end to end workflow или бизнес-сценарии. Рекомендуется написание тестовых сценариев или тестовых случаев, чтобы охватить различные сценарии и установить критерии прохождения;